CANIS: designing a conditioning robot that a frightened dog can live with

CREST Gold Award project report

Supporting documents submitted alongside this report: `TIMELINE.md` (plan and time management) · `RISK_ASSESSMENT.md` (safety and welfare) · `../paper/canis_paper_draft_v2.md` (full systems paper) · `../BUILD_STORY.md` (build log) · `../PROBLEMS.md` (problem and risk inventory) · `../../hardware/TESTING.md` (dated bench-validation results).

> **Scope, stated up front.** This project is submitted as a **systems, design and safety engineering** study. It makes **no claim** that the device changes the dog's behaviour, and no behavioural trial was run. That scoping is deliberate and is argued in §12. A behaviour-change claim would require a controlled, multi-subject, ethically-reviewed study, and reporting one without that would be the single worst thing this project could do. Results reported here are engineering-validation results, and they're real, dated, and logged.
>
> **Author:** Siddharth. No project mentor. Remaining self-checks are collected in §14.

---

1. Aim and objectives

Aim: to design and build a home robot that helps condition a dog's behaviour through consistent, precisely-timed reward, for a dog who is *afraid of moving objects*, which makes the robot itself the problem the design has to solve.

The subject is a young Goldendoodle who shows several target behaviours (eating things he shouldn't, barking, climbing furniture, roaming) and is measurably frightened by movement. Most automated pet devices would make this dog's situation worse, not better. That single fact reframes the whole engineering problem: robot motion isn't a free actuator, it's a **hazard to be rationed**.

Five objectives followed:

#	Objective	Success measure	Outcome
O1	Deliver reward on a conditioning-relevant timescale	A marker cue plus treat, triggered by a detected behaviour, with delivery confirmed rather than	**Met.** Confirmed-ACK dispensi correlation ids (§9)

#	Objective	Success measure	Outcome
		assumed	
O2	Never punish; reward the desired alternative	No punishment path exists in the state machine at all	**Met by construction** (§3)
O3	Make robot motion safe by architecture, not by care	The robot halts within ~1 s of losing its command link, in every failure mode	**Met and bench-verified** (§8.4
O4	Make welfare a controllable, default-safe quantity	A single explicit exposure level, defaulting to no motion, reversible, with the system still useful at that default	**Met.** `EXPOSURE_LEVEL` (§8
O5	Identify the dog reliably enough that a false reward is rare	Near-deterministic identification, no false triggering on other people or pets	**Met.** ArUco fiducial on the col (§8.3), bench-verified end to e

A sixth objective, running the gradual-exposure protocol to the point of logged behavioural observations, was **not met**, and §11 says why plainly.

---

2. Wider purpose

Companion-animal behaviour problems are common, and the reason most owners fail to resolve them isn't indifference but timing: effective conditioning needs consistent reinforcement delivered within a second or two of the behaviour, every time, and humans just aren't reliable at that. An always-present device is genuinely attractive here.

The dominant risk is equally real. An automated "corrector" easily becomes an aversive, frightening presence, and for an animal already sensitive to movement, a machine that approaches when it disapproves is close to the worst possible intervention. The wider purpose of this project is to show that the two can be separated: that you can build a device which conditions behaviour *without* the device becoming something the animal fears.

The generalisable claim is about where welfare lives in a system. In most designs welfare is diffuse, a collection of good intentions spread across the code, each of which can be individually eroded by a later change. CANIS concentrates it into one explicit, default-safe control (§8.5). That's a transferable pattern for any system acting physically on a living thing, and it's the part of this work that would matter to someone building something else.

3. Approaches considered

Punish entry, rejected on welfare grounds. The obvious design, detect the dog in a forbidden zone and deter it, is the standard approach and was rejected before any code was written. For a fearful dog, punishing zone entry pairs the zone *and the robot* with fear, producing an animal who's frightened of a machine that lives in his house. The alternative adopted is to **mark and reward the dog for leaving**: the desired alternative behaviour is reinforced, and disengagement is what earns the treat. Because the reward is contingent on the act of leaving, the system has no punishment path to be misconfigured into.

YOLO or a learned object detector, rejected on reliability. The instinct for "find the dog" is a CNN. Fiducial markers, an ArUco `4x4_50` tag on the collar, were chosen instead, for three reasons argued in paper §3.6. The task isn't "find any dog in any pose" but "is *this* dog in *this* room," and a fiducial gives a near-deterministic, low-false-positive answer. That matters more than usual here, because **a false correction is actively harmful to a fearful animal**. The cost of a false positive is asymmetric in a way that ordinary detection benchmarks don't capture. A CNN would also consume far more of the Pi's compute budget for no reliability gain on a single, cooperative, marker-wearing subject. And the marker *is* identity, so other people or pets in frame trigger nothing.

This is framed as a pragmatism result, not a general claim that fiducials beat learned detectors. For a reliability-critical, single-subject, compute-constrained home deployment, the fiducial is the correct engineering choice; change any of those three conditions and the answer changes.

A single controller, rejected on failure isolation. Running everything on one board is simpler. It was rejected because it couples failure domains that need to stay separate: vision stalls would delay motor commands, and a crash would take the safety watchdog down with the thing it's watching. The three-tier split (§8.2) exists so the real-time executor can't be starved by a network stall.

A printed chassis, abandoned mid-project. Four CAD revisions in, an available Meccano-style perforated metal chassis proved simpler and stronger than the printed deck. The pivot is described in §11.

---

4. Plan and rationale

Full detail in `TIMELINE.md`. The constraint *do no harm to a fearful animal* inverted the conventional build order: the stop path, the motion watchdog, and the exposure gate were built and tested **before** the robot was allowed to move near the dog. "Get it driving, add safety later" was never an available plan here.

Subsystems were validated in isolation before integration, in the order specified by `hardware/TESTING.md`: motors, dispenser, sensors, inter-board link, vision, power, then the full loop. With three processors and two link protocols, a failure in an integrated system is expensive to localise, and an unvalidated subsystem operating near a frightened animal is a welfare risk, not merely a bug.

5. Time management

The project ran **2026-06-26 to 2026-07-19**, 24 days, **84 hours** across seven phases, with a real two-week pause between the public release (Jul 7) and the chassis pivot (Jul 19). `TIMELINE.md` gives the dated phase table and Gantt, traced to commits and to the dated results log in `hardware/TESTING.md`.

Two departures from plan:

CAD cost far more than budgeted. Four revisions, each with a mechanical interference that appeared only once printed, at a full print cycle per attempt.

Phase 7 wasn't planned at all. The chassis pivot forced a re-derivation of the mounting layout, wiring plan, and assembly order that had all assumed a printed deck.

---

6. Resources and people

Three processors, each matched to what it does best. An unusual amount of hardware for a school project, justified by failure isolation rather than capability. An Arduino Uno as the real-time executor ("limbs") for jitter-free servo and motor timing, kept deliberately dumb and reliable. An ESP32 ("brain") running the arousal state machine and coordinating both other tiers. A Raspberry Pi 5 ("eyes and face") for OpenCV vision and the Flask dashboard, which also owns the wall clock.

Working with what was available. The most useful resource decision was abandoning four revisions of printed-chassis CAD when a perforated metal chassis turned out to be simpler and stronger. That meant accepting that the castor's 24 mm bolt circle and the dispenser's three feet at 25 mm radius, 120° apart, don't land on a 10 mm square grid cleanly. Resolved by bolting whichever foot lands on a real hole and zip-tying the rest, recorded honestly rather than pretending the grid and the CAD agree.

People, stated accurately, including the gap. This was a solo build, and the honest position is that the expertise this project most needs was *identified but not engaged*: paper §8 names prior review by an animal-research ethics body and/or a qualified veterinary behaviourist as a requirement before any efficacy claim could be made. That review hasn't been sought, which is precisely why the project is

scoped to make no such claim. Knowing which expert you'd need, and scoping your claims to what you can support without them, is the responsible version of working alone. No mentor was used for this project. It was built independently throughout, and the profile form's mentor field is left blank accordingly.

The subject. The dog is a family pet, and his participation is bounded by supervision, the welfare gate, and an owner who can end any session at will (§12).

---

7. Background research

Three sources ground the specific claims; full citations in §16, discussion in paper §2.

The conditioning model rests on established practice in using reward to change a fear response *without flooding* the animal [1]. Flooding, forced exposure to the feared stimulus, is exactly what a naively-designed robot does, and the gradual-exposure architecture (§8.5) exists to avoid it. The staged, criterion-gated structure of the exposure protocol follows the standardised training-program model in [2]. The perception layer uses the ArUco fiducial detection algorithm directly [3], which also supplies the justification for the ArUco-over-YOLO argument in §3.

> **To strengthen (criterion 2.2):** three citations is adequate but thin against LYRA's six. Two additions would harden it materially: a source on canine fear-response and body-language coding, which the exposure protocol needs anyway to define its advance/hold/retreat criteria, and one on reinforcement timing and delay-of-reward, which bears directly on the ~5 to 9 s reward latency identified as a validity risk in `PROBLEMS.md` §H.

---

8. System design

8.1 Design constraints

Three constraints drove every later decision (paper §3.1):

- **C1, do no harm to a fearful animal.** Motion is a hazard to be rationed. This makes "stops reliably and quickly" a safety requirement, and "functions while not moving at all" a first-class operating mode.
- **C2, reward leaving, never punish entering.**
- **C3, empirical, tunable thresholds**, named and co-located in firmware and in the dashboard simulator, and documented as provisional rather than settled.

8.2 Three-tier compute split

Device	Role	Why here
Arduino Uno, "limbs"	Real-time executor: motors, dispenser servo, marker buzzer	Jitter-free 5 V timing; deliberately dumb and reliable
ESP32, "brain"	Sensors, arousal state machine, decisions, both links	Enough compute for the state machine plus W coordinate
Raspberry Pi 5, "eyes and face"	OpenCV vision, Flask dashboard	Vision and a web UI exceed either MCU; the P owns the wall clock

The split isolates failure domains. The real-time executor can't be starved by a network stall, and the decision tier *polls* the Pi rather than being driven synchronously by it, so vision going away degrades the system instead of blocking it.

8.3 Detection

Two independent sources. Locally, the ESP32 fuses a PIR sensor with three ultrasonic rangefinders (left, centre, right), fired sequentially to avoid echo cross-talk, the trio doubling as a coarse bearing. Remotely, the Pi detects the collar's ArUco marker and posts a *zone label*, not imagery, which the ESP32 polls. Frame-skipping and a Laplacian-variance blur guard reject motion-blurred frames rather than classifying them.

Ultrasonics give range, not identity; they can't distinguish the dog from a wall. This is why, while moving, the dog is identified by the camera marker *only*, a constraint honestly recorded in `PROBLEMS.md` §A rather than glossed over.

8.4 The arousal state machine

A scalar **arousal** level in [0, 100] drives three states, `CALM`, `AROUSAL`, `CORRECTION`, with **rate-based hysteresis**: separate enter and exit thresholds, so arousal has to climb clearly past one line to turn a state on and fall clearly past a lower line to turn it off. Without hysteresis, a dog sitting exactly at a threshold made the state flip every 20 ms, re-firing the buzzer dozens of times for one continuous event (§9).

Two numerical safeguards protect the arousal integration. `dt` is clamped to 0.2 s per tick, so a single stalled loop iteration can't make `rise-rate × dt` teleport the system from calm to a full correction in one step. And the vision latch clears itself on a local 4-second timeout rather than waiting for a network round-trip (§9).

Fail-safe motion. The real-time Arduino runs a motion watchdog and halts within ~1 s of its command link going silent. A crashed ESP32, a dead router, or a pulled cable therefore all fail to *stopped*, never to *moving toward the dog*. Motors also stop on any reset.

8.5 The gradual-exposure gate: welfare as one knob

`EXPOSURE_LEVEL` concentrates welfare into a single explicit control, described in the code as "the welfare knob." It **defaults to 0, no movement at all**, and the system remains fully useful there, delivering marker and reward with no motion whatsoever. Levels advance one step at a time, only when the dog's logged fear response shows tolerance, and every step is reversible. Escalation requires a two-key arm-then-confirm rather than one keypress, and exposure resets to 0 on boot.

The architectural point is that this **decouples what the robot decides from how much it's allowed to physically move.** The decision logic can be developed, debugged, and run at full capability while the physical envelope stays at zero.

---

9. Problems identified and overcome

A latch that could never clear. The ESP32 decided "is the dog here?" partly from a vision flag latched in `visionActive`, which was only ever *cleared* by a successful poll returning "no dog." If WiFi dropped, or the Pi crashed, or the router rebooted, the clearing poll never arrived and the latch stuck on **forever**: the buzzer firing every two seconds, and at higher exposure levels the robot re-orienting toward a dog that might have left an hour earlier. This was the worst behaviour the system was capable of, and it was reachable by the most common home-network failure there is. Fixed with a local 4-second TTL that clears the latch without any network round-trip, and mirrored in the test suite so it can't regress silently.

Chatter at the threshold. A dog sitting at exactly the correction threshold made the state flip every 20 ms, re-firing the buzzer and inflating intervention counts dozens of times for one continuous event. Fixed with hysteresis.

The safety guarantee that ordinary arithmetic could violate. At the lowest "safe" exposure level, orient only, never approach, the bearing logic could legitimately compute FORWARD as the direction to face the dog. The Arduino has no concept of "face" versus "drive at": FORWARD is FORWARD. The one guarantee the entire project rested on could be broken by correct bearing arithmetic rather than by a bug in a rare branch. Fixed by rewriting any FORWARD bearing below the approach-enabled level into an in-place pivot.

Treats counted that never fell. The dispenser counted a treat as given the moment the ESP32 *asked* for

one. A jam or an empty hopper still rang the marker tone and still incremented the count. For a system whose entire mechanism is "sound predicts treat," a marker with nothing behind it doesn't merely waste a session, it teaches the dog the marker is a lie. Fixed twice over: delivery confirmed back from the Arduino so rewards count on ACK not on request, and a correlation id per dispense so a coincidental manual button press can't be miscredited to a pending automatic reward. After two consecutive jams the dispenser goes silent rather than keep beeping over an empty hopper.

CAD that looked right and didn't fit. Three print revisions each carried a real interference: the treat chute ran straight through the castor wheel's position; the funnel's sloped wall was clipped by the shelf cutout; the castor spacer overlapped the deck skirt by 7 mm; the hopper lid was modelled as a straight plug pushed into a funnel that keeps narrowing, so it couldn't seat the last 2.5 mm. Every one looked correct in a 3D preview and only failed once printed in plastic, a full print cycle per discovery. The fix that mattered is in §11.

---

10. Results, engineering validation

Per-subsystem bench tests, executed and logged. **Results log dated 2026-07-06** in `hardware/TESTING.md`:

Test	Result
1, Motors (L298N, both channels PWM-matched)	**PASS.** Forward, back, pivot, and stop all correct
2, Dispenser	**DEFERRED.** Mechanism works; `GATE_OPEN_MS` tuning happens once mounte the chassis, not on the bench
3, Sensors	**PASS.** All three ultrasonics plus PIR clean
4, ESP32 to Arduino link	**PASS**
5, Three-way Pi to ESP32 to Arduino comms	**PASS.** Full communication confirmed

Additionally verified on hardware (paper §4.1): vision end to end, camera to OpenCV/ArUco detecting marker ID 0 to directional zone posted to dashboard serving it to the polling ESP32; the state machine's real outputs including dispense, correction cue, reward cooldown and cap, and watchdog-safe pulsed orienting; and the dashboard's live link with simulator fallback, auto-starting on boot.

What was measured, and what wasn't. These are engineering-validation results: whether each subsystem behaves as specified, whether the closed loop makes the intended transitions, whether the fail-safes fire. **No controlled behavioural trial was run,** and none of the data a behaviour-change claim requires (baseline behaviour rates, a control condition, blinded coding, multiple subjects) was collected. That's a deliberate scoping decision, not an omission (§12).

---

11. Creative thinking

Inverting the conditioning primitive. The creative core of the project is recognising that the standard design isn't merely suboptimal for a fearful dog but actively counterproductive, and that inverting it, reward leaving rather than punish entering, solves the welfare problem and the engineering problem at once. It removes the punishment path entirely, so there's no aversive behaviour left in the system to be reached by a misconfiguration.

Making welfare a single quantity. Turning a diffuse design intention into one explicit, default-safe, reversible control is the pattern most likely to transfer to something else. It converts "we were careful" into a property that can be inspected, tested, defaulted safe, and reset on boot.

Turning fit into a computation. The most useful methodological invention came out of the CAD failures. Instead of eyeballing a preview, every part's real assembly position is checked by asking OpenSCAD to compute the literal 3D intersection of the two placed solids: two parts that don't touch intersect to nothing, two parts that overlap intersect to a solid you can see and measure. That converts "does this fit?" from a rendering judgement into a yes/no computation, and it now gates every dimension the project claims anywhere. The general form matters more than the trick: when your eye keeps getting a class of question wrong, stop trying to look harder and find a way to *compute* the answer.

Naming the failure mode you're actually in. `PROBLEMS.md` records the project's honest top risk as *"build perfectionism means it never actually runs with the dog,"* alongside the instruction to ship the simplest working mode and observe. Writing that down about your own project, in your own repository, is uncomfortable, and that's what makes the rest of the document trustworthy. See §13.4.

---

12. Ethics and welfare

The full register is in `RISK_ASSESSMENT.md`, with animal-welfare hazards listed first because constraint C1 outranks capability everywhere in the system.

Scope. No claim is made that CANIS changes the dog's behaviour. No institutional animal-ethics approval was obtained, deliberately, because no behavioural result is being reported. If the work were ever reframed toward an efficacy claim, prior review from an animal-research ethics body and/or a qualified veterinary behaviourist would be required *first*, and the exposure protocol's fear-response coding scheme would need pre-registering rather than being defined after the data was in. Choosing the scope your evidence supports, rather than the scope that sounds most impressive, is itself an ethical decision.

Welfare-first design. Default no movement; a mild auditory cue and a food reward; reinforcement of a desired alternative rather than positive punishment; motion introduced one reversible step at a time, gated on the dog's logged response.

Physical safety. Hardware motion watchdog halting within ~1 s of any command-link loss; correction motions duration-bounded and halted immediately on disengagement; `NEVER_APPROACH` in every mode; speeds dialled down for exposure sessions; supervision throughout, with the session ended immediately on any sign of distress **or** over-arousal. The second is watched for as closely as the first, since a chased robot can wind a dog up rather than frighten it.

Reward safety. 20 s cooldown and a 30-per-session cap, enforced in all modes with a compile-time guard, preventing over-feeding and hopper depletion. Treats are a vet-recommended training treat; the count given varies with session length rather than being fixed, and the daily total is confirmed to sit comfortably within a safe allowance.

Data and privacy. The camera performs on-device fiducial detection only: a zone label is posted, not imagery. Recorded data is sensor counters, arousal and state logs, and marker detections. The vision script has a `--snap` diagnostic flag that can save a still frame to disk; whether it's ever been used outside a one-off bring-up test, and what retention policy applies if so, is confirmed by me in §14. The home network SSID was found in tracked test sketches and scrubbed, including from the audit file that had recorded it, and a provenance convention was added so stale claims can't drift unnoticed.

Open welfare items, stated as open. The beam-break jam detector has no mount and ships disabled, so a silent jam is caught only by gate-cycle counting. Reward latency of ~5 to 9 s may reinforce whatever the dog did in the interval; the immediate marker tone is the standard bridge, but latency reduction is open work. (Over-arousal, previously listed here, is resolved above: the stop-on-over-arousal rule is an explicit, confirmed protocol, not an outstanding gap.)

Consent. The animal's owner (me) consents to participation and can end any session at will.

---

13. Conclusions, implications, and reflection

13.1 Conclusions

A conditioning robot for a fear-sensitive dog can be built such that welfare is an architectural property rather than a matter of operator care. Three specific conclusions are supported by the validation in §10:

1. **Fail-safe motion is achievable with cheap hardware.** Splitting the real-time executor from the decision tier and giving the executor an independent watchdog means every failure path (crashed controller, dead network, pulled cable) terminates in *stopped*. Verified on the bench.
2. **Welfare can be reduced to one controllable quantity.** `EXPOSURE_LEVEL` decouples decision-making from physical envelope, so the full decision logic can run and be debugged while the robot isn't permitted to move at all.
3. **A fiducial beats a learned detector under this cost asymmetry.** When a false positive causes real harm to a frightened animal, near-deterministic identification is worth more than generality, and it costs less compute.

All five design objectives in §1 were met. The sixth, behavioural observations, wasn't, and no behavioural claim is made anywhere in this report.

13.2 Implications

The transferable result is about **where welfare lives in a system that acts physically on a living thing.** The default engineering approach spreads care across many decisions, each individually erodable by a later change. Concentrating it into a single explicit, default-safe, reversible, boot-resetting control makes it inspectable and testable, and means someone can verify the safety property without reading the whole codebase. That applies well beyond dogs: assistive robotics, care equipment, anything sharing space with someone who can't consent in the ordinary sense.

The second implication is about detection cost asymmetry. Detection systems are benchmarked on symmetric metrics, but here a false positive harms the animal while a false negative merely misses a training opportunity. When the costs are that asymmetric, the correct architecture may be the less sophisticated one.

13.3 How my decisions shaped the outcome

Choosing reward-on-leaving over punishment determined everything downstream. It's why there's no aversive path in the system to be reached by any misconfiguration, and it turned a welfare requirement into a structural property.

Building the stop path before the drive path is why the FORWARD-bearing violation was caught. The

guarantee was explicit and being checked, so it was found by inspection rather than by the robot doing something visibly wrong near a frightened dog.

Choosing a fiducial over a CNN traded generality for determinism, and it was the right trade only because of the cost asymmetry. I wouldn't make the same choice for a system where a missed detection was the expensive outcome.

Scoping the work as design rather than efficacy cost the more impressive claim and is the reason every number in this report can be defended.

The through-line: each of these decisions accepted a smaller claim or a less capable system in exchange for a property that could be verified.

13.4 What I learnt, and what I would improve

The technical learning spanned three fields that rarely meet: control and real-time systems (watchdogs, hysteresis, failure-domain isolation, numerical safeguards on integration), computer vision (fiducial detection, blur rejection), and animal-behaviour science (counter-conditioning, reward timing, the difference between reinforcing an alternative and suppressing a behaviour).

The larger lesson was about honesty as an engineering practice. Two moments taught it. The dispenser counting treats that never fell was a case where the system's *internal* dishonesty had a direct external cost: a marker with nothing behind it teaches the dog the marker is a lie. And writing down that the project's dominant risk was my own build perfectionism was uncomfortable in exactly the way accurate self-assessment is.

Four improvements, in priority order:

1. **Run the exposure protocol.** This is the real gap. Level 0 is buzzer-only with no motion, it was runnable from Phase 1, and it needed nothing from the chassis. I sequenced the project as though the behavioural work depended on the finished robot, and at Level 0 it never did. Weeks of logged fear-response data would exist by now had I started it in parallel.
2. **Define the fear-response coding scheme before collecting data.** Latency to approach, tail and ear posture, avoidance distance, willingness to take the treat, with pre-registered advance/hold/retreat criteria. Defining these after seeing the data would make them worthless.
3. **Close the open welfare items.** Mount and enable the beam-break jam detector, and give over-arousal a robot-side signal rather than relying on supervised judgement alone to catch it.
4. **Reduce reward latency.** At ~5 to 9 s the marker tone is doing all the bridging work; shortening the loop would make the conditioning mechanism sounder.

---

14. Author checks before submission

Resolved: hours (84), mentor (none, §6), stop-session criteria (ends immediately on any sign of distress or over-arousal, §12), and treat budget (vet-recommended treat, daily total confirmed to fit safely, count varies by session, §12).

Genuinely open, and only I can close this:

1. **Camera retention.** Confirm the `--snap` diagnostic has never written frames to disk outside the one bring-up test noted in §12, or state the retention policy if it has. This depends on how the device has actually been run, which no amount of reading the tracked launcher script settles.
2. **Two more citations (§7), optional.** Canine fear-response coding and reward-delay timing. Both are needed for the exposure protocol anyway, but this is a strengthening, not a blocker.
3. **Do not submit the paper draft with `[Author TODO]` markers in it.** Paper §5 and §8 carry unresolved placeholders. This report supersedes §5 by scoping to engineering validation; if the paper is attached as a supporting document, resolve or remove them first. An assessor reads a bracketed TODO as abandoned work.

---

15. Criteria map

#	Criterion	Where evidenced
1.1	Clear aim, broken into objectives	§1, incl. the O1–O5 table
1.2	Wider purpose	§2
1.3	Range of approaches identified	§3, four alternatives, each rejected on a stated ground
1.4	Plan described, with rationale	§4; `TIMELINE.md` §1
1.5	Time planned and organised well	§5; `TIMELINE.md` §2 to 4 (dated phases + Gantt)
2.1	Good use of materials and people	§6, incl. the honest account of expertise identified but not engaged
2.2	Research and referencing	§7, §16; paper §2
3.1	Logical conclusions + wider	§13.1, §13.2, conclusions drawn from §10's validation results

#	Criterion	Where evidenced
	implications	
3.2	How actions and decisions affected the outcome	§13.3
3.3	What was learnt, and improvements	§13.4
4.1	Understanding of the science	§8, hysteresis, failure-domain isolation, numerical safeguards, fiducial dete counter-conditioning
4.2	Ethical and safety decisions, incl. risk assessment	§12; `RISK_ASSESSMENT.md`
4.3	Creative thinking	§11
4.4	Problems identified and overcome	§9
4.5	Explained clearly	This report throughout

---

16. References

[1] S. R. Lindsay, *Handbook of Applied Dog Behavior and Training, Volume One: Adaptation and Learning*. Ames, IA: Iowa State University Press, 2000. ISBN 978-0-8138-0754-6.

[2] A. Stellato, S. Jajou, C. E. Dewey, T. M. Widowski, and L. Niel, "Effect of a Standardized Four-Week Desensitization and Counter-Conditioning Training Program on Pre-Existing Veterinary Fear in Companion Dogs," *Animals*, vol. 9, no. 10, p. 767, 2019. doi: 10.3390/ani9100767.

[3] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion," *Pattern Recognition*, vol. 47, no. 6, pp. 2280 to 2292, 2014. doi: 10.1016/j.patcog.2014.01.005.